Sesiones de refuerzo: programación y lógica

Carlos A Delgado S
Pontificia Universidad Javeriana Cali

De qué se trata

Dieciséis horas de refuerzo repartidas en cuatro bloques de cuatro horas. Cada bloque dedica tres horas a programación y una a lógica.

Los conceptos de programación se dan por vistos en Estructuras de Datos. Cada hora los recuerda en diez a quince minutos, con lo que cuesta cada operación, y dedica el resto a resolver en el computador un problema de juez en línea donde ese concepto hace falta. Las horas cuyo tema ya se repasó dedican ese tiempo al problema. La línea empieza en arreglos y matrices, sigue con el conteo de operaciones y la recursión, pasa por pilas, colas, diccionarios y montículos, y termina en búsqueda binaria y recorrido en amplitud.

La línea de lógica repasa lo que hace falta para leer y escribir una demostración: proposiciones, cuantificadores, la inducción aplicada a las fórmulas del conteo, a las funciones recursivas y a sus recurrencias, los tipos de demostración y, al final, las dos que el curso usa todo el tiempo, la cota con O grande y el invariante de ciclo.

Al terminar, el estudiante elige la estructura que un problema pide y sabe cuánto le cuesta cada operación. Los temas del curso dan esa base por vista.

Bloque	**Hora**	**Tema**	**Problema en clase**
1	1	Arreglos: un recorrido contra dos	UVa 11078, Open Credit System
	2	El mejor tramo de un arreglo	UVa 507, Jill Rides Again
	3	Matrices y acumulados	UVa 108, Maximum Sum
	4	Lógica: proposiciones y predicados	en papel
2	5	Conteo de operaciones	UVa 12149, Feynman
	6	Recursión: casos y árbol de llamadas	UVa 839, Not so Mobile
	7	Recursión por índices y su costo	UVa 11173, Grey Codes
	8	Lógica: inducción y recurrencias	en papel
3	9	Pilas y colas en simulación	UVa 12100, Printer Queue
	10	Diccionarios y conjuntos	UVa 156, Ananagrams
	11	Montículos y colas de prioridad	UVa 1203, Argus
	12	Lógica: tipos de demostración y cotas con O	en papel
4	13	Búsqueda binaria y estructuras ordenadas	UVa 10611, The Playboy Chimp
	14	Bisección sobre la respuesta	UVa 11413, Fill the Containers
	15	Grafos y recorrido en amplitud	UVa 924, Spreading the News
	16	Lógica: invariantes de ciclo	en papel

Horarios

El primer bloque abre la serie. Al terminarlo se acuerdan con los asistentes los horarios de los tres que siguen, con un máximo de dos bloques por semana: entre uno y otro hay que resolver los ejercicios y dejar que lo visto se asiente.

Cómo se trabaja un bloque

- Cada hora de programación tiene dos tramos. Los primeros diez a quince minutos repasan el concepto y lo que cuesta cada operación. El resto de la hora se resuelve en el computador el problema de esa hora: se escribe, se ejecuta y se prueba con los casos del enunciado. Si el tema ya se repasó, ese tiempo pasa al problema.
- Los lenguajes son Python y C++. Quien viene de Estructuras de Datos puede resolver el mismo ejercicio en C.
- Solo biblioteca estándar, sin `break`, sin `continue` y con un solo `return` al final de cada función.
- La hora de lógica se trabaja en papel.
- Cada bloque deja tres problemas de tarea. Se resuelven antes del bloque siguiente, se envían al juez y se revisan en vivo en sus primeros minutos.

Programación por horas

Las horas 5, 9 y 13 empiezan con la revisión de la tarea del bloque anterior. Después siguen la forma de las demás: repaso breve y problema en el computador.

El bloque 1 abrió con un repaso de las estructuras con su costo y su implementación: arreglo dinámico y lista, lista enlazada, deque, pila, cola, direccionamiento directo y tabla hash, árbol balanceado, montículo con inserción y extracción, iteradores y memoria, y la pila de llamadas del factorial seguida en el depurador. Después se resolvió UVa 11078. Quedan del bloque 1 las horas 2, 3 y 4, que se trabajan en ese orden antes de la hora 5: UVa 507, UVa 108 y la lógica de proposiciones y predicados. Como las estructuras ya se repasaron, las horas 9, 10 y 11 recuerdan en dos minutos lo que cuesta cada operación, remiten a ese repaso por tema y dan el tiempo al problema; la hora 6 parte de la traza de la pila de llamadas.

Hora	Repaso	Problema en el computador
Bloque 1. Arreglos y matrices		
1	Acceso por posición en $O(1)$ y recorrido en $\Theta(n)$. Cuándo un doble ciclo de $\Theta(n^2)$ se cambia por un recorrido que lleva lo mejor visto hasta ahí.	UVa 11078, *Open Credit System*: la mayor diferencia $a_i - a_j$ con $i < j$, en un solo recorrido.
2	El mejor tramo que termina en cada posición y cómo sale del anterior: la idea de Kadane.	UVa 507, *Jill Rides Again*: el tramo de ruta de mayor suma, con su desempate.
3	Matrices: representación, recorrido por filas y por columnas, y acumulados por columna que dan la suma de un rango en $O(1)$.	UVa 108, *Maximum Sum*: fijar dos filas y reducir al caso de un arreglo.
4	Lógica, en papel: proposiciones y conectivos; la implicación con su recíproca y su contrarrecíproca; predicados y cuantificadores con su dominio. Traducir a fórmula propiedades del mundo real (todo inscrito tiene nota, ningún horario se cruza con otro) y matemáticas (el arreglo está ordenado, un valor aparece en un rango, la definición de O).	
Bloque 2. Conteo y recursión		

Hora	Repaso	Problema en el computador
5	Revisión de la tarea. Contar línea a línea cuántas veces se ejecuta cada instrucción y escribir el total como $T(n)$: ciclos simples, con paso distinto de 1, dependientes del índice externo y con un índice que se duplica; las sumatorias cerradas $\sum 1$, $\sum i$ y $\sum i^2$; mejor y peor caso.	UVa 12149, *Feynman*: cuántos cuadrados hay en una cuadrícula de $N \times N$; el conteo del triple ciclo es la respuesta y su fórmula cerrada la da sin ciclos.
6	De la traza de la pila de llamadas del factorial al árbol de llamadas: caso base, caso recursivo y una rama por cada llamada.	UVa 839, *Not so Mobile*: decidir si un móvil con submóviles está en equilibrio.
7	Recursión por índices: la función recibe *ini* y fin y no copia. Plantear la recurrencia del programa a partir de cuántas llamadas hace y cuánto trabaja fuera de ellas: el conteo de la hora 5 cuando la función se llama a sí misma.	UVa 11173, *Grey Codes*: el código de la posición k sin construir la lista de 2^n códigos.
8	Lógica, en papel: las fórmulas de las sumatorias de la hora 5 demostradas por inducción; correctitud de una función recursiva por inducción (base, hipótesis y paso); plantear $T(n)$ de un programa recursivo; resolver $T(n) = T(n-1) + c$, $T(n) = T(n/2) + c$ y $T(n) = 2T(n/2) + cn$ por sustitución y con el árbol de recursión, y comprobar la solución por inducción.	
Bloque 3. Pilas, colas, diccionarios y montículos		
9	Revisión de la tarea. Costos de pila y cola en dos minutos, según el repaso de estructuras: sacar del frente de una lista de Python cuesta $\Theta(n)$; sacar de una cola, $O(1)$.	UVa 12100, *Printer Queue*: simular la cola de impresión hasta que sale el trabajo propio.
10	Costos de diccionario y conjunto en dos minutos, según el repaso de estructuras: buscar, insertar y borrar en $O(1)$ promedio. Una clave que representa a toda una clase de objetos.	UVa 156, *Ananagrams*: las palabras que no tienen anagrama en el diccionario.
11	Costos del montículo en dos minutos, según el repaso de estructuras: insertar y extraer en $O(\log n)$. Cola de prioridad con pares: `heapq` con tuplas en Python, `priority_queue` con comparador en C++.	UVa 1203, *Argus*: las primeras K respuestas de un grupo de consultas periódicas.
12	Lógica, en papel: teorema, estrategia, desarrollo y conclusión; demostración directa, por contrarrecíproca y por contradicción, con un ejemplo numérico y uno sobre arreglos; demostrar una cota de O grande exhibiendo los testigos c y k.	
Bloque 4. Búsqueda binaria, bisección y entrada a grafos		
13	Revisión de la tarea. Búsqueda binaria sobre $[l..r)$ y la primera posición que cumple una condición; `bisect` en Python, `lower_bound` y `upper_bound` en C++.	UVa 10611, *The Playboy Chimp*: el vecino menor y el mayor de cada consulta en un arreglo ordenado con repetidos.
14	De optimización a decisión: la función objetivo, su monotonía y la bisección sobre la respuesta.	UVa 11413, *Fill the Containers*: la menor capacidad máxima de m contenedores.

Hora	Repaso	Problema en el computador
15	Lista de adyacencia y recorrido en amplitud con una cola, en $O(V+E)$; la distancia por niveles.	UVa 924, *Spreading the News*: el día en que más empleados se enteran por primera vez.
16	Lógica, en papel: invariantes de ciclo escritos como fórmula; inicialización, estabilidad y terminación sobre un ciclo de acumulación y sobre la búsqueda binaria.	

Nivel de los ejercicios

El nivel de referencia son las tareas del curso de Estructuras de Datos y la primera tarea de Árboles y Grafos. Los problemas de clase son de ese nivel, y los tres de tarea de cada bloque están escalados:

- **Base.** el tema del bloque aplicado directo.
- **Intermedio.** el tema más un formato de entrada o de salida que hay que respetar al pie de la letra.
- **Exigente.** el tema con una decisión de diseño adentro, sea la estructura o el costo.

La tarea se resuelve entre un bloque y el siguiente, y su revisión ocupa los primeros minutos del bloque siguiente. Los problemas salen de UVa, salvo uno de la tarea del bloque 3, que es de HackerRank. La dirección del enunciado y la del envío van escritas completas, porque el ejercicio no termina hasta que el juez responde.

Bloque 1. Arreglos y matrices

Programación (tres horas)

Hora 1. UVa 11078, *Open Credit System*.
https://onlinejudge.org/external/110/11078.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2019

Se busca el mayor valor de $a_i - a_j$ con $i < j$, donde el índice menor es el estudiante más antiguo. El parámetro es la posición j: para cada una, la mejor resta usa el máximo de $a_0, \ldots, a_{j-1}$, y ese máximo se actualiza al avanzar. Basta un recorrido. El doble ciclo revisa unos 5×10^9 pares en un caso con $n = 100\,000$, y la entrada trae varios. La trampa está en la muestra: con una secuencia creciente la respuesta es -1, así que el máximo arranca en la primera resta y no en 0.

Hora 2. UVa 507, *Jill Rides Again*.
https://onlinejudge.org/external/5/507.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=448

Cada ruta trae $s - 1$ valores, uno por tramo entre paradas, y se pide el tramo continuo de mayor suma. El parámetro es el valor donde termina el tramo. El mejor que acaba ahí extiende al que acababa en el anterior, salvo cuando esa suma ya es negativa: entonces conviene empezar de nuevo. Se reinicia con suma negativa y no con cero, porque un prefijo de suma cero alarga el tramo sin cambiar su valor, y el enunciado desempata por el tramo más largo y después por el que empieza antes. Por eso el mejor se actualiza cuando la suma es mayor o cuando, con suma igual, el tramo es más largo. Las paradas se numeran desde 1: el tramo que va del valor i al j se imprime entre las paradas i y $j + 1$. Si la mejor suma no es positiva, la salida es `Route r has no nice parts`.

Hora 3. UVa 108, *Maximum Sum*.
https://onlinejudge.org/external/1/108.pdf

Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=44

La matriz es de $N \times N$, con $N \leq 100$, y se pide el subrectángulo de mayor suma. Probar cada rectángulo con acumulados en dos dimensiones cuesta $O(N^4)$. Lo que baja el costo es fijar el par de filas, la de arriba y la de abajo. Con ellas fijas, la suma de cada columna entre las dos filas forma un arreglo de N valores, y el mejor rectángulo con esas filas es el mejor tramo de ese arreglo: el problema de la hora 2. Los acumulados por columna dan cada suma en $O(1)$ y el total queda en $O(N^3)$. Hay dos trampas. Los N^2 números llegan repartidos en líneas cualesquiera, así que se leen de corrido sin contar líneas. Y si todo es negativo la respuesta es negativa, porque el rectángulo tiene al menos una celda.

Lógica (una hora)

Proposiciones y conectivos. La implicación con su recíproca y su contrarrecíproca. Cuantificadores con su dominio y el rango de los índices.

El ejercicio de la hora es traducir propiedades a fórmula: que todos los elementos del arreglo son positivos, que el arreglo está ordenado, que un valor aparece en un rango. Con eso se lee la definición que el curso usa desde la primera clase: $f \in O(g)$ cuando existen $c > 0$ y $k \geq 0$ tales que $f(n) \leq c\, g(n)$ para todo $n \geq k$.

Tarea

Base. UVa 100, *The 3n + 1 problem.*
https://onlinejudge.org/external/1/100.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=36

Un ciclo aplica la regla hasta llegar a 1 y cuenta los números que imprime; el recorrido del rango se queda con el máximo. La longitud cuenta el 1, así que la de 22 es 16. La trampa es que i puede llegar mayor que j: el recorrido va del menor al mayor, pero la salida repite i y j en el orden en que llegaron. Los valores intermedios pasan de 10 000, así que no sirven como posición de un arreglo del tamaño de la entrada.

Intermedio. UVa 10189, *Minesweeper.*
https://onlinejudge.org/external/101/10189.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1130

Por cada celda sin mina se cuentan las minas de sus ocho vecinas, con los dos desplazamientos recorriendo $-1, 0, 1$ y la comprobación de borde. El primer número son las filas y el segundo las columnas. La trampa está en la salida: la línea en blanco va entre campos y no después del último, y las minas se conservan tal como llegaron.

Exigente. UVa 10920, *Spiral Tap.*
https://onlinejudge.org/external/109/10920.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1861

Aquí la matriz no se puede construir: con un lado de hasta 100 000 son 10^{10} celdas. Se trabaja por anillos, cada uno con su rango de posiciones, y adentro del anillo la posición se convierte en fila y columna con aritmética. La trampa es el tamaño: la posición llega hasta el lado al cuadrado y se pasa de 32 bits. La fila 1 es la de abajo y el centro de la espiral es la posición 1.

Bloque 2. Conteo y recursión

Programación (tres horas)

Hora 5. Conteo de operaciones.

Al frente de cada línea se anota cuántas veces se ejecuta y el total, como función del tamaño de la entrada, es $T(n)$. La condición de un ciclo se cuenta cada vez que se evalúa, también la última, la que da falso: $n+1$ veces para n vueltas. Con `while` se cuentan por separado la condición, el cuerpo y el incremento; con `for` la cabecera cuenta como la condición. Las constantes cambian entre las dos maneras y la clase no. Cuando el ciclo interno depende del externo, se congela el índice externo, se cuentan las vueltas del interno como función de él y se suman con las fórmulas cerradas: $\sum_{i=0}^{n-1} 1 = n$, $\sum_{i=0}^{n-1} i = n(n-1)/2$ y $\sum_{i=0}^{n-1} i^2 = (n-1)n(2n-1)/6$. Cada $T(n)$ se comprueba contando a mano con un n pequeño.

En papel, de menor a mayor: sumar los elementos de un arreglo; buscar un valor con y sin detenerse al encontrarlo, con su mejor y su peor caso; un índice que se duplica hasta pasar de n, que da $\lfloor \log_2 n \rfloor + 1$ vueltas; sumar una matriz completa y solo su triángulo superior, donde el ciclo interno arranca en i; y tres bloques en secuencia, que se suman.

UVa 12149, *Feynman*.
https://onlinejudge.org/external/121/12149.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3301

Se pide cuántos cuadrados hay en una cuadrícula de $N \times N$. La forma directa es un triple ciclo: el lado l de 1 a N y, para cada lado, la fila y la columna de la esquina, de 0 a $N-l$. Contar cuántas veces corre el cuerpo es contar los cuadrados: $\sum_{l=1}^{N}(N-l+1)^2 = \sum_{m=1}^{N} m^2 = N(N+1)(2N+1)/6$. Con $N \leq 100$ el ciclo alcanza, y la fórmula da la respuesta en $O(1)$; las dos se comparan sobre todos los N antes de enviar. Para $N = 8$ son 204. La trampa está en el límite de la esquina: va hasta $N-l$ incluido, y con $N-l$ excluido la muestra da 140 en vez de 204. La entrada termina con un 0 que no se procesa.

Hora 6. UVa 839, *Not so Mobile*.
https://onlinejudge.org/external/8/839.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=780

Cada línea describe una barra con dos extremos, peso y distancia de cada lado, y la barra está en equilibrio cuando $W_l \times D_l = W_r \times D_r$. Un peso 0 significa que de ese extremo cuelga un submóvil, descrito en las líneas que siguen. Ese es el caso recursivo; el extremo con objeto es el caso base. La función lee una línea, se llama a sí misma por cada extremo con peso 0, primero el izquierdo y después el derecho, y devuelve dos cosas: el peso total del submóvil, que reemplaza al 0 de la barra de arriba, y si todo lo que cuelga de ella está en equilibrio. El árbol de llamadas es el mismo móvil. La trampa está en la lectura: aunque un submóvil ya esté desequilibrado, la función tiene que leer todas sus líneas, o el caso siguiente arranca en mitad de este. La entrada trae el número de casos y líneas en blanco entre ellos, y la salida separa los casos con una línea en blanco que no va después del último.

Hora 7. UVa 11173, *Grey Codes*.
https://onlinejudge.org/external/111/11173.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2114

El código reflejado de n bits se arma con el de $n-1$: arriba, en orden y con un 0 delante; abajo, reflejado y con un 1 delante. La recursión trabaja sobre el rango de posiciones $[ini, fin)$, que empieza en $[0, 2^n)$. Si k cae en la mitad de arriba, el bit es 0 y se sigue con esa mitad; si cae en la de abajo, el bit es 1 y se sigue con la posición reflejada, porque esa mitad repite la de arriba al revés. Cada llamada hace una sola llamada sobre la mitad del rango y trabajo constante, así que $T(m) = T(m/2) + c$ con $m = 2^n$ posiciones: $n+1$ llamadas por caso, la primera incluida. Construir la lista no se puede, porque

con $n = 30$ son más de mil millones de códigos. La trampa está en el reflejo: en la mitad de abajo la posición k pasa a ser $fin - 1 - k$ dentro de la mitad de arriba, no k menos la mitad, y las posiciones se cuentan desde 0, como muestra la tabla del enunciado. Con hasta 250 000 casos, la lectura y la escritura tienen que ser rápidas.

Lógica (una hora)

La hora empieza por las fórmulas de la hora 5. Que $\sum_{i=0}^{n-1} i = n(n-1)/2$ se demuestra por inducción sobre n, y la de $\sum i^2$ sale igual. Es la misma demostración que se usa después con las funciones recursivas.

Después viene la parte matemática de la recursión. La correctitud de una función recursiva se demuestra por inducción sobre el tamaño de la entrada: el caso base de la función es la base de la inducción, y la hipótesis supone que las llamadas sobre entradas más pequeñas devuelven lo correcto. Con eso el paso solo tiene que mirar lo que la función hace con esas respuestas.

El costo sale de la misma lectura. Se plantea $T(n)$ con cuántas llamadas hace el programa, sobre qué tamaño y cuánto trabaja fuera de ellas, y se resuelven las tres recurrencias del curso: $T(n) = T(n-1) + c$, $T(n) = T(n/2) + c$ y $T(n) = 2T(n/2) + cn$. Cada una por sustitución y con el árbol de recursión, y la solución se comprueba por inducción. Las de las horas 6 y 7 sirven de ejemplo.

Tarea

Base. UVa 10079, *Pizza Cutting*.
https://onlinejudge.org/external/100/10079.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1020

Se pide el mayor número de pedazos que dejan N cortes rectos. El corte k cruza a los $k-1$ anteriores y pasa por k pedazos, así que agrega k: el total es $1 + \sum_{k=1}^{N} k = 1 + N(N+1)/2$. El ciclo que suma corta N vueltas por caso, y N llega a 210 000 000; la fórmula da cada respuesta en $O(1)$. La trampa es el tamaño: el resultado pasa de 32 bits. Con $N = 0$ hay un pedazo, y la entrada termina con cualquier número negativo, no solo con -1.

Intermedio. UVa 913, *Joana and the Odd Numbers*.
https://onlinejudge.org/external/9/913.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=854

La fila con N números es la $L = (N+1)/2$, y las filas anteriores tienen $1, 3, 5, \ldots$ números. Contar cuántos impares hay escritos hasta el final de esa fila es una sumatoria: $\sum_{i=1}^{L}(2i-1) = L^2$. El último impar es entonces $2L^2 - 1$ y los tres últimos suman $6L^2 - 9$: para $N = 3$ son 15, como en la muestra. Recorrer las filas una por una no alcanza, porque N llega a 10^9. La trampa es de nuevo el tamaño: $6L^2$ se pasa de 32 bits, y el enunciado garantiza que cabe en 64.

Exigente. UVa 620, *Cellular Structure*.
https://onlinejudge.org/external/6/620.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=561

Una cadena sana es `A`, o una sana seguida de `AB`, o `B` seguida de una sana y de `A`. La función recibe el rango $[ini, fin)$ de la cadena y decide la etapa: el caso base es la cadena de una letra, y los dos casos recursivos llaman sobre el rango sin las dos letras del final o sin la primera y la última. La última letra decide cuál de los dos casos se intenta, `B` para el primero y `A` para el segundo, así que cada nivel hace a lo más una llamada: $T(n) = T(n-2) + c$, lineal. Cortar la cadena en cada llamada copia n letras por nivel y deja el costo en $\Theta(n^2)$. La trampa está en qué etapa se imprime: la de la última regla aplicada, la de afuera, mientras que lo de adentro solo tiene que estar sano, en cualquier etapa. `BAABA` es `MUTAGENIC`

porque `AAB` es sana, y `BAAB` es `MUTANT` porque termina en `B` y `BA` no es sana. Una cadena de largo par nunca es sana.

Bloque 3. Pilas, colas, diccionarios y montículos

Programación (tres horas)

Hora 9. UVa 12100, *Printer Queue*.
https://onlinejudge.org/external/121/12100.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3252

La cola trae hasta 100 trabajos con prioridad de 1 a 9, y se pregunta en qué minuto sale el trabajo de la posición m. Cada elemento de la cola es un par: la prioridad y una marca que dice si es el trabajo propio. La marca evita seguir un índice que cambia con cada vuelta. En cada paso la pregunta es si alguien en la cola tiene prioridad mayor que el primero, y nueve contadores, uno por prioridad, la responden sin recorrer la cola. Tres trampas salen del enunciado: la posición m se cuenta desde 0; un trabajo de igual prioridad no obliga a mover el primero; y los minutos cuentan solo las impresiones, incluida la del trabajo propio.

Hora 10. UVa 156, *Ananagrams*.
https://onlinejudge.org/external/1/156.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=92

Una palabra es ananagrama si ninguna otra del diccionario se forma con sus mismas letras. La clave de cada palabra son sus letras en minúscula y ordenadas, porque dos palabras son anagramas si y solo si tienen la misma clave. Un diccionario de clave a número de palabras se llena en un recorrido, y un segundo recorrido imprime las palabras cuya clave aparece una sola vez. La trampa es doble. La comparación ignora mayúsculas, pero la salida imprime cada palabra como llegó y las ordena distinguiéndolas: en la muestra, `Disk` y `NotE` van antes de `derail`. Además, las palabras de una letra son ananagramas, una línea trae varias palabras con espacios libres y la entrada termina en una línea con `#`.

Hora 11. UVa 1203, *Argus*.
https://onlinejudge.org/external/12/1203.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3644

El montículo vive en un arreglo: los hijos de la casilla i están en $2i + 1$ y $2i + 2$, y subir o bajar un valor cuesta $O(\log n)$. Aquí se usa el del lenguaje, con pares.

Cada consulta registrada responde a los P segundos de registrarse y después cada P segundos; se piden las primeras K respuestas. La cola de prioridad guarda un par por consulta, el instante de su próxima respuesta y su número. Sale la de menor par, y vuelve a entrar con el instante aumentado en su periodo. El orden de los pares resuelve por sí solo el desempate que pide el enunciado: si dos consultas responden en el mismo instante, sale primero la de número menor. Las trampas son de lectura. La primera respuesta llega a los P segundos, no en el instante 0; el registro termina en una línea con `#`; y K llega a 10 000 con a lo más 1000 consultas.

Lógica (una hora)

Toda demostración del curso se escribe en cuatro partes: el teorema, la estrategia que se va a seguir, el desarrollo y la conclusión.

Con esa forma se hacen las tres estrategias de la hora: directa, contrarrecíproca y contradicción, con un ejemplo numérico y uno sobre arreglos por cada una. La comparación entre la contrarrecíproca y la recíproca es donde se pierde media demostración.

Después, una cota de O grande demostrada en las mismas cuatro partes, exhibiendo los testigos c y k. Donde CLRS escribe n_0, aquí va k.

Tarea

Base. UVa 11286, *Conformity.*
https://onlinejudge.org/external/112/11286.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2261

Cada estudiante elige cinco cursos, y la combinación es el conjunto, no el orden en que llegan: la primera y la tercera línea de la muestra son la misma. La clave es la lista de cursos ordenada, y un diccionario de clave a número de estudiantes da la popularidad de cada combinación. La trampa está en lo que se imprime. No es la mayor popularidad ni el número de combinaciones, sino el total de estudiantes que toman alguna combinación de popularidad máxima: si todas son distintas, como en el segundo caso de la muestra, la respuesta es el número de estudiantes. La entrada termina con 0.

Intermedio. UVa 10954, *Add All.*
https://onlinejudge.org/external/109/10954.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1895

Cada suma cuesta lo que suman sus dos operandos, así que conviene sumar siempre los dos menores. La cola de prioridad entrega esos dos en tiempo logarítmico y recibe de vuelta el resultado, que vuelve a competir con los demás. Por eso ordenar una vez y sumar en fila no alcanza: ahí está el ejercicio. La trampa es el acumulado, que se pasa de 32 bits.

Exigente. HackerRank, *QHEAP1.*
Enunciado y envío: https://www.hackerrank.com/challenges/qheap1/problem

Tres operaciones sobre un conjunto de enteros: agregar un valor, borrar un valor y consultar el mínimo. Agregar y consultar son las de siempre. Borrar obliga a abrir el montículo, porque el valor puede estar en cualquier posición y `heapq` solo saca la raíz. Si se sabe que el valor está en la posición p, se cambia por el último, se acorta el arreglo y se repara desde p, subiendo o bajando según el caso, en $O(\log n)$. Un diccionario de valor a posición dice dónde está, y cada intercambio del montículo lo actualiza. Eso funciona porque el enunciado garantiza que en todo momento los valores son distintos. Buscar el valor recorriendo el arreglo cuesta $\Theta(n)$ por borrado, con hasta 10^5 operaciones. El borrado perezoso es la otra salida: marcar el valor y descartarlo cuando llegue a la raíz. Los valores van de -10^9 a 10^9, negativos incluidos.

Bloque 4. Búsqueda binaria, bisección y entrada a grafos

Programación (tres horas)

Hora 13. UVa 10611, *The Playboy Chimp.*
https://onlinejudge.org/external/106/10611.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1552

Las N alturas llegan en orden no decreciente, y para cada consulta q se pide la más alta menor que q y la más baja mayor que q. Una altura igual a q no sirve para ninguna de las dos. Dos búsquedas binarias dan los bordes: la primera posición con altura $\geq q$ y la primera con altura $> q$. La anterior a la primera es la más alta menor; la segunda es la más baja mayor. Lo que permite buscar así es el orden, porque cada condición es falsa en un prefijo del arreglo y verdadera en el resto. Con $N = 50\,000$ y $Q = 25\,000$, recorrer el arreglo en cada consulta da unos $1{,}25 \times 10^9$ pasos. La trampa son los repetidos y los bordes: cuando una de las dos alturas no existe, se imprime `X` en su lugar.

Hora 14. UVa 11413, *Fill the Containers*.
https://onlinejudge.org/external/114/11413.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2408

Los recipientes de la banda se vacían en orden en m contenedores, cada recipiente entero en uno solo, y se pide la menor capacidad posible del contenedor más grande. En vez de buscar el óptimo se pregunta si alcanza con capacidad t. Un recorrido voraz lo responde: llena el contenedor actual mientras quepa el recipiente siguiente, abre otro cuando no cabe, y t alcanza si usa a lo más m. Si t alcanza, cualquier capacidad mayor también, y esa monotonía es la que hace funcionar la bisección entre el recipiente más grande y la suma de todos. La trampa está en los rangos: m llega a 10^6 con a lo más 1000 recipientes, así que pueden sobrar contenedores, y la cota inferior de la bisección es el recipiente más grande, porque ninguno se divide. Hay varios casos hasta fin de archivo.

Hora 15. UVa 924, *Spreading the News*.
https://onlinejudge.org/external/9/924.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=865

Cada empleado cuenta la noticia a sus amigos al día siguiente de enterarse, y se pide el día en que más empleados se enteran por primera vez y cuántos son. El día en que un empleado se entera es su distancia a la fuente en el grafo de amistades, y el recorrido en amplitud asigna esas distancias por niveles: basta contar cuántos quedan en cada nivel. La red se lee una vez y sirve para todos los casos, así que las distancias se reinician con cada fuente. Las trampas son tres. Los amigos de la fuente se enteran el día 1; ante un empate en el tamaño gana el primer día; y si nadie más que la fuente se entera, la salida es un solo `0`.

Lógica (una hora)

Un invariante se escribe como fórmula, con su cuantificador, sus índices y el rango de cada uno; la lectura en palabras viene después, para decirlo en voz alta. Una frase no se puede sustituir en $j + 1$ ni evaluar al final del ciclo, y eso es justo lo que la demostración necesita.

Las tres partes, inicialización, estabilidad y terminación, sobre dos ciclos: uno de acumulación y la búsqueda binaria de la hora 13.

Tarea

Base. UVa 10341, *Solve It*.
https://onlinejudge.org/external/103/10341.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1282

La función del enunciado es decreciente en $[0, 1]$ porque cada término lo es, con los signos que el enunciado impone a los coeficientes. Esa monotonía es lo que hace funcionar la bisección: si la función en 0 es negativa o en 1 es positiva, no hay raíz en el intervalo. La trampa es la comparación con cero, que va con tolerancia, y la salida, que pide cuatro decimales. La entrada termina en fin de archivo, sin centinela.

Intermedio. UVa 336, *A Node Too Far*.
https://onlinejudge.org/external/3/336.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=272

El TTL se cuenta en saltos, así que el recorrido en amplitud desde el nodo de la consulta responde directo: se cuentan los nodos cuya distancia pasa del TTL y los que no se alcanzan. Las etiquetas no son $0, 1, \ldots, n - 1$, así que hace falta un diccionario de etiqueta a índice antes de armar la lista de

adyacencia. La trampa es el contador de casos, que es global y no se reinicia con cada red, y el texto de la salida, que dice `nodes` también cuando es uno solo.

Exigente. UVa 714, *Copying Books*.
https://onlinejudge.org/external/7/714.pdf
Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=655

Es el problema de la hora 14 con una exigencia más: imprimir la partición. La pregunta que se bisecta es de decisión: ¿alcanza con que ningún escriba copie más de t páginas? Es monótona en t, y un recorrido voraz que abre escriba nuevo cuando se pasa de t la responde. La bisección devuelve el menor t que sí alcanza. Las trampas son tres: entre las particiones óptimas se pide la que deje menos trabajo al primer escriba, que se consigue armando los grupos desde el final; las sumas llegan a 5×10^9 y se pasan de 32 bits; y cada escriba lleva al menos un libro.

Lo que hay que traer

- Computador con Python 3 y un compilador de C++ instalados y probados antes del bloque.
- Cuenta en el juez UVa, https://onlinejudge.org/, y en HackerRank, https://www.hackerrank.com/, para la hora 10.
- Los ejercicios del bloque anterior intentados. El que no compila sirve igual: es el que se revisa.
- Papel para la hora de lógica.

Bibliografía

- T. Cormen, C. Leiserson, R. Rivest, C. Stein. *Introduction to Algorithms*. Tercera edición, MIT Press, 2009. Capítulos 2 y 3: invariantes y O grande.
- K. Rosen. *Matemáticas Discretas y sus Aplicaciones*. McGraw-Hill. Lógica, demostración e inducción.
- R. Thareja. *Data Structures Using C*. Oxford University Press, 2018.
- S. Halim, F. Halim, S. Effendy. *Competitive Programming 4*, 2020. Problemas de juez por tema.